

Шпаргалка и конспект: сбор банковских данных, web scraping и backend

Сводка всего диалога. Фокус: практическая модель задачи подключения банков, scraping, Playwright/BeautifulSoup, parser, устойчивость и ключевые backend-тезисы для интервью.

1. Главный вывод: что значит «подключить банк»

В практическом смысле новый банк часто означает новый source-specific адаптер/парсер. Это не обязательно новый сервис. Общая инфраструктура может быть общей, а для нового источника реализуется логика извлечения и нормализации данных.

Банк → получить данные → parse_data(...) → нормализованные данные → validation → общий pipeline

Ключевое различие: «отдельный парсер» ≠ «отдельный сервис». Хорошая архитектура переиспользует fetch/validation/storage/monitoring и изолирует различия конкретного банка.

2. Где нужна web/frontend-экспертиза

Для каждого банка надо определить, где реально находятся нужные данные: исходный HTML, embedded JSON, API, вызываемый JavaScript, или только DOM после выполнения JavaScript. Для этого нужны DOM, CSS-селекторы, DevTools Network, понимание JS-rendering и browser automation.

3. Алгоритм выбора инструмента

Сначала проверяем обычный HTTP response. Если данные уже в HTML — requests + BeautifulSoup. Если данные в JSON — JSON parser. Если JavaScript получает данные из API — по возможности ходим напрямую в API. Если без выполнения JavaScript/browser context данные недоступны — Playwright.

Главный принцип: Playwright нужен не потому, что сайт написан на React, а потому, что обычного HTTP-запроса недостаточно для получения нужных данных.

4. requests / HTTP

```
response = requests.get(url, timeout=10)
html = response.text
```

Первый вопрос: какой документ реально пришёл? HTTP 200 сам по себе не означает, что получена нужная страница.

5. BeautifulSoup и DOM

BeautifulSoup разбирает HTML и позволяет искать элементы. CSS-селекторы выбирают нужные узлы. Предпочтительны стабильные признаки: id, data-атрибуты, семантические признаки и устойчивая структура. Хрупкие nth-child-селекторы лучше избегать.

```
soup = BeautifulSoup(html, "html.parser")
element = soup.select_one(".offer .rate")
text = element.get_text(strip=True)
```

6. Конкретный пример Сбера

В присланном HTML курс уже присутствовал в response body:

```
<div class="Teaser402-block__rates-values">
  <p>EUR 106.2</p>
  <p>USD 89.7</p>
</div>
```

Следовательно, для этого блока Playwright не нужен: данные можно извлечь непосредственно из HTML.

```
elements = soup.select(".Teaser402-block__rates-values p")
```

```
for element in elements:
    print(element.get_text(" ", strip=True))
```

Наличие reactdom.js и teaser402.js не меняет этот вывод: наличие JavaScript-файлов само по себе не означает, что данные генерируются только в браузере.

7. Если данных в HTML нет

Типичный сценарий: сервер отдаёт контейнер, JavaScript запускается, делает fetch/XHR к API, получает JSON и вставляет данные в DOM. В DevTools: F12 → Network → reload → Fetch/XHR → найти запрос, возвращающий нужные данные.

GET /page
↓
JavaScript
↓
GET /api/rates
↓
JSON
↓
DOM

Если найден прямой API, его обычно выгоднее использовать, чем поднимать полноценный браузер.

8. Playwright

Playwright запускает браузерный движок, выполняет JavaScript и позволяет получить итоговый DOM. После этого можно работать с locator'ами или передать page.content() в BeautifulSoup.

Playwright → Chromium → JS → итоговый DOM → parser

Playwright дороже по CPU/RAM и сложнее в эксплуатации, поэтому его выбирают по необходимости.

9. Backend-граница: parse_data(html) -> dict

Если инфраструктура получения страницы уже существует, source-specific backend-логика действительно может быть очень простой:

```
def parse_data(html: str) -> dict:
    soup = BeautifulSoup(html, "html.parser")
    ...
    return {"EUR": ..., "USD": ...}
```

Ответственность parser'a: HTML/JSON → структурированные данные. HTTP, Kafka, PostgreSQL и прочая инфраструктура лучше отделены от parser'a.

10. Устойчивость parser'a

Стабильность обеспечивается системой вокруг parser'a: стабильные селекторы; обязательные поля; schema/domain validation; явные ошибки; метрики; regression fixtures; alerts при изменении структуры.

HTML → parser → validation
↓
ошибка / alert
или
↓
нормальные данные

Принцип: лучше громко обнаружить изменение сайта, чем тихо записать неправильный курс.

11. Тестирование

Реальные HTML-ответы сохраняются как fixtures. Тесты проверяют извлечение ожидаемой модели. Это позволяет обнаруживать регрессии без обращения к живому сайту.

fixtures/
sber_2026_09_01.html

```
test:
result = parse_data(load_fixture(...))
assert result["USD"] == Decimal("89.70")
```

12. Общая архитектура scraper pipeline

Fetcher / Browser
↓
source-specific parser
↓
normalized data
↓
validation
↓
diff / persistence
↓
Kafka / DB / API
↓
monitoring

13. Kafka — важные тезисы

Partition — единица параллелизма и порядка внутри topic. Consumer group распределяет partitions между consumers. Максимум одновременно активных consumers для topic в группе ограничен числом partitions.

Порядок гарантируется только внутри partition.

Key помогает направлять связанные сообщения в одну partition и тем самым сохранять порядок для данного ключа. Offset показывает позицию consumer'a. Если процесс упал после обработки, но до commit, сообщение может быть обработано повторно — обработчики должны учитывать idempotency.

Kafka также полезна как буфер между сбором и обработкой и позволяет независимо масштабировать этапы.

14. PostgreSQL / SQL — что было изучено

Нужно понимать реляционную модель, индексы, транзакции, блокировки, нормализацию, cardinality и планы запросов. EXPLAIN/EXPLAIN ANALYZE связывают выбранный план с реальными затратами.

ANALYZE собирает статистику для планировщика. Ошибочная или устаревшая статистика может привести к плохому плану; важны распределение значений и n_distinct.

Для конкурентных обновлений важны транзакции и блокировки. SELECT FOR UPDATE в SQLAlchemy рассматривается в контексте транзакции.

15. TLS и почему браузер работает, а requests нет

В диалоге requests получил SSLCertVerificationError: self-signed certificate in certificate chain. После verify=False удалось получить HTML, но это оказалась fallback/error page Сбера про сертификаты, а не нужная страница с курсами.

```
requests
↓
TLS verification
↓
CA не доверен
↓
SSLCertVerificationError
```

Браузер и Python могут использовать разные trust stores и разные характеристики клиента. Поэтому браузер может работать, а requests — нет.

verify=False — только диагностический эксперимент. В production проверку TLS отключать нельзя.

В production при использовании собственного CA/TLS проху сертификат CA должен быть корректно установлен в trust store контейнера/ОС или явно передан HTTP-клиенту.

16. «Не тот HTML» и «сломался parser» — разные проблемы

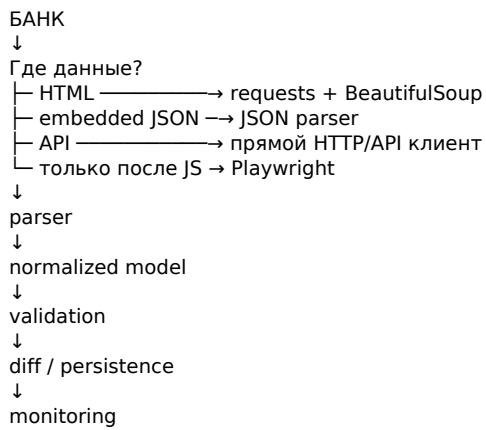
```
requests.get()
↓
какой документ реально получен?
↓
если error/fallback page
↓
parser ни при чём
```

Перед изменением CSS-селектора проверяем status code, final URL, длину response, наличие ожидаемого фрагмента и начало response.text.

17. Интервью: короткие ответы

Вопрос	Короткий ответ
Как подключить новый банк?	Сделать source-specific адаптер извлечения, а общую инфраструктуру получения, валидации, хранения и мониторинга переиспользовать.
Когда нужен Playwright?	Когда обычного HTTP-запроса недостаточно: данные появляются после JS, нужен browser context или сервер иначе обрабатывает browser-клиент. Если есть прямой API — по возможности использовать API.
Зачем BeautifulSoup?	Разобрать HTML и извлечь нужные элементы по устойчивым селекторам.
Как сделать parser устойчивым?	Стабильные селекторы + validation + regression fixtures + метрики и alerts.
HTTP 200 означает успешный scraper?	Нет. Нужно проверить, что получен ожидаемый документ и обязательные данные извлечены и прошли validation.
Почему браузер работает, а requests нет?	Браузер и Python могут использовать разные trust stores и client characteristics. В конкретном случае requests столкнулся с TLS/CA проблемой и получил fallback page.
Что делать первым при подключении банка?	Определить источник данных и только после этого выбрать requests, прямой API или Playwright.

18. Финальная ментальная модель



Фраза для запоминания: «Сначала определяю источник данных, затем выбираю способ получения, после чего source-specific parser приводит данные к единой модели; результат валидируется и мониторится».